



Funciones y módulos

Clase

Codificar una rutina utilizando funciones preconstruidas, funciones personalizadas o de un módulo para resolver un problema de baja complejidad de acuerdo al lenguaje python.

- Unidad 1: El lenguaje Python
- Unidad 2: Sentencias básicas del lenguaje Python
- Unidad 3: Sentencias condicionales
- Unidad 4: Funciones y módulos
- Unidad 5: Estructura de datos en Python
- Unidad 6: Secuencias iterativas
- Unidad 7: Conceptos básicos de orientación a objeto en Python



Te encuentras aquí



¿Qué aprenderás en esta sesión?

- *Reconoce las principales funciones preconstruidas disponibles en el lenguaje python.*
- *Elabora funciones personalizadas con parámetros y retorno de acuerdo a la sintaxis del lenguaje python para resolver el problema planteado.*
- *Utiliza funciones pertenecientes a un módulo para resolver un problema de baja complejidad.*

En las sesiones anteriores,
ya hemos estado utilizando
funciones sin profundizar
en su definición,
¿Recuerdas cuáles?



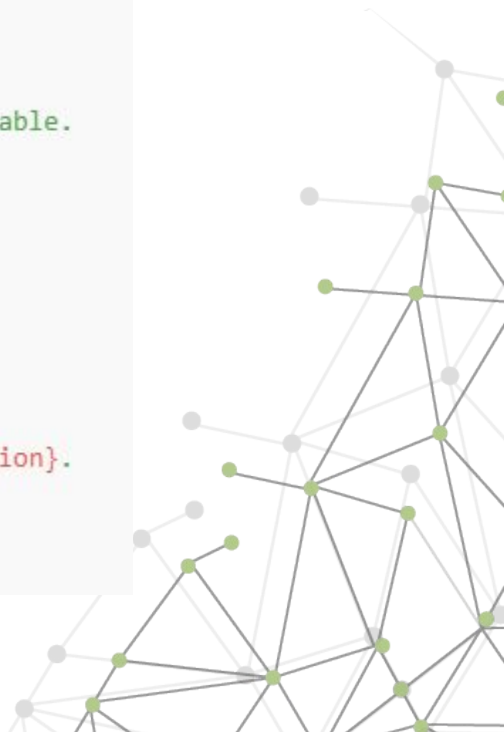
Recordemos el ejercicio “Presentándome con Python”

```
nombre = input('Ingrese su Nombre: ')\nedad = input('Ingrese Edad: ')\nocupacion = input('Ingrese Ocupación: ')\nprint(f'''\nMi nombre es {nombre}, tengo {edad} años y trabajo como {ocupacion}.\n''')
```



Ahora podríamos mejorarlo usando una función

```
def crear_presentacion():  
    """  
    Crea una presentación personal basada en los datos ingresados.  
    Esta función mejora nuestro código anterior haciéndolo reutilizable.  
    """  
  
    nombre = input('Ingrese su Nombre: ')  
    edad = input('Ingrese Edad: ')  
    ocupacion = input('Ingrese Ocupación: ')  
  
    presentacion = f'''  
    Mi nombre es {nombre}, tengo {edad} años y trabajo como {ocupacion}.  
    ...  
    '''  
  
    return presentacion
```



`/* Funciones */`

Funciones

¿Qué son y para qué sirven?

Imaginemos una cocina profesional. Cuando el chef principal prepara un plato especial, sigue una serie de pasos (una receta) que puede reutilizar cada vez que alguien pide ese plato.

No tiene que reinventar la receta cada vez; simplemente sigue los pasos establecidos, tal vez con pequeñas variaciones según los ingredientes disponibles.

Funciones

¿Qué son y para qué sirven?

En programación, una función es similar a una receta:

- Es un bloque de código organizado y reutilizable.
- Realiza una tarea específica.
- Puede recibir datos (ingredientes).
- Puede devolver resultados (el plato terminado).

Ventajas de usar funciones:

- Reutilización de código.
- Organización y claridad.
- Mantenimiento más fácil.
- Prevención de errores.

Funciones preconstruidas en python

- Las funciones preconstruidas (o built-in functions) son como una caja de herramientas que Python nos proporciona desde el inicio.
- Imagina que compras un teléfono nuevo:
 - viene con aplicaciones básicas preinstaladas (calculadora, calendario, reloj, etc.).
- De la misma manera, Python viene con funciones básicas listas para usar.

Además de las funciones de entrada **input()** y de salida **print()** que ya conocemos, existen otras funciones reconstruidas:

- De conversión;
- De análisis;
- Matemáticas básicas;
- De ayuda.

Funciones preconstruidas en python

Funciones de conversión de tipos

Imagina que estás en un país extranjero donde se hablan diferentes idiomas. Para comunicarte, necesitas traductores que te ayuden a convertir las palabras de un idioma a otro.

En Python, las funciones de conversión son como estos traductores: nos permiten transformar datos de un tipo a otro para que puedan "comunicarse" y trabajar juntos.

Las principales funciones de conversión son:

- `int()`
- `float()`
- `str()`

Funciones preconstruidas en python

Funciones de conversión de tipos

- **int()**: Convierte a números enteros.

```
año_texto = input("¿En qué año naciste? ") # Usuario ingresa "2000"  
año = int(año_texto) # Convierte "2000" a 2000  
edad = 2024 - año    # Ahora podemos hacer operaciones matemáticas  
print(f"Tienes {edad} años.")
```

Funciones preconstruidas en python

Funciones de conversión de tipos

- **float():** Convierte a decimales.

```
peso_texto = input("Peso en kg: ")    # Usuario ingresa "70.5"
altura_texto = input("Altura en m: ") # Usuario ingresa "1.75"

peso = float(peso_texto)    # Convierte a 70.5
altura = float(altura_texto) # Convierte a 1.75
imc = peso / (altura ** 2)

print(f"Tu IMC es: {imc:.2f}")
```

Funciones preconstruidas en python

Funciones de conversión de tipos

- **str():** Convierte a texto.

```
# Ejemplo práctico: Formato de número de teléfono
codigo_area = 56
numero = 12345678
telefono = str(codigo_area) + str(numero)
# Resultado: "5612345678"
print(f"El número de teléfono es: {telefono}")
```

Funciones preconstruidas en python

Funciones de análisis

Imagina que eres un detective y tienes una caja misteriosa. Antes de abrirla, necesitas saber:

- ¿qué hay dentro?, ¿qué tamaño tiene?, ¿es seguro abrirla?

Las funciones de análisis en Python son como las herramientas de un detective: nos ayudan a investigar y entender nuestros datos antes de trabajar con ellos.

Las principales funciones de análisis son:

- `type()`
- `len()`

Funciones preconstruidas en python

Funciones de análisis

- **type()** es como un escáner que nos dice exactamente qué tipo de dato tenemos. Nos ayuda a prevenir errores antes de que ocurran.

```
# Imagina una aplicación de supermercado:
precio = "1500" # El precio viene como texto desde el código de barras

# Verificamos el tipo
print(f"El precio es de tipo: {type(precio)}") # Resultado: <class 'str'>

# ¡Ups! No podemos hacer cálculos con texto
# Necesitamos convertirlo a número
if type(precio) == str:
    precio = int(precio)

# Ahora sí podemos calcular el descuento
descuento = precio * 0.1
print(f"El descuento es: ${descuento:.2f}")
```


Funciones preconstruidas en python

Funciones de análisis

- **len()** es como una regla que mide la longitud o cantidad de elementos. Es útil para validar datos y tomar decisiones.

```
# Verificar seguridad de La contraseña
if len(contraseña) < 8:
    print("Error: La contraseña debe tener al menos 8 caracteres")
    return False

print("Datos válidos - Usuario registrado")
return True
```

¡Manos a la obra! Inspector de recetas de cocina 👩🍳



Inspector de recetas de cocina

Ejercicio guiado

Eres chef principal de un restaurante famoso y necesitas un programa que te ayude a analizar las recetas que tus cocineros/as están ingresando al sistema. Es importante verificar que los nombres de las recetas sean correctos y tengan la longitud adecuada para el menú.



Inspector de recetas de cocina



Ejercicio guiado

Antes de comenzar el ejercicio, necesitamos primero introducir el concepto de "def". La palabra "def" (de "define") es como poner un cartel que dice "aquí comienza una nueva función".

Es similar a cuando escribes una receta: primero pones el título (nombre de la función) y luego los pasos a seguir (el código).

```
def nombre_de_la_funcion():  
    # Aquí va el código de la función  
    # Todo lo que esté indentado pertenece a la función
```

Ya conoceremos cómo crear una función.



Inspector de recetas de cocina

Ejercicio guiado

1. Crear el archivo inspector_recetas.py
2. Crear una función simple.

```
# Creamos nuestra primera función  
def revisar_receta():  
    print("=== INSPECTOR DE RECETAS ===")  
    print("Analizando receta...")
```

Inspector de recetas de cocina



Ejercicio guiado

3. Agregar la entrada de usuario.

```
def revisar_receta():  
    print("=== INSPECTOR DE RECETAS ===")  
    nombre = input("Ingresa el nombre de la receta: ")  
    print(f"Analizando la receta: {nombre}")
```

4. Agregar un análisis básico.

```
def revisar_receta():  
    print("=== INSPECTOR DE RECETAS ===")  
    nombre = input("Ingresa el nombre de la receta: ")  
  
    # Revisamos si está vacío  
    if len(nombre) == 0:  
        print("❌ Error: La receta necesita un nombre")  
    else:  
        print(f"✅ Nombre registrado: {nombre}")
```

Funciones preconstruidas en python

Funciones matemáticas básicas

Imagina que tienes una caja de herramientas matemáticas. Así como tienes una regla para medir, una calculadora para operaciones básicas y un compás para círculos, Python viene con sus propias herramientas matemáticas listas para usar.

Las principales funciones matemáticas son:

- `round()`
- `abs()`
- `max()` y `min()`

Funciones preconstruidas en python

Funciones matemáticas básicas

- **round()** es útil para redondear números decimales:
 - round(numero) redondea a entero.
 - round(numero, 2) redondea a 2 decimales.

```
precio = 4.6666
precio_redondeado = round(precio, 2) # Resultado: 4.67
print(f"Precio redondeado: {precio_redondeado}")
```


Funciones preconstruidas en python

Funciones matemáticas básicas

- **abs()** obtiene el valor absoluto.
 - Convierte números negativos a positivos.
 - Los números positivos se mantienen igual.

```
temperatura_mañana = 15
temperatura_tarde = 22

# ¿Cuánto cambió la temperatura?
cambio = abs(temperatura_mañana - temperatura_tarde)
print(f"La temperatura cambió {cambio} grados")
```

Funciones preconstruidas en python

Funciones matemáticas básicas

- **max() y min()** comparan valores.
 - max() encuentra el valor más alto.
 - min() encuentra el valor más bajo.

```
# Ejemplo - Precios de un producto
precio_tienda1 = 15000
precio_tienda2 = 12000
precio_tienda3 = 18000

# Encontrar el mejor precio
mejor_precio = min(precio_tienda1, precio_tienda2, precio_tienda3)
print(f"El mejor precio es ${mejor_precio}")

# Encontrar el precio más alto
precio_mayor = max(precio_tienda1, precio_tienda2, precio_tienda3)
print(f"El precio más alto es ${precio_mayor}")
```

¡Manos a la obra! Calculadora de notas



Calculadora de notas

Ejercicio

Eres un profesor que necesita calcular las notas finales de sus estudiantes. Para hacer este proceso más eficiente, crearás un programa que ayude a calcular promedios, identificar la nota más alta y más baja, y redondear los resultados adecuadamente.

¿Cómo lo lograrías?



Funciones personalizadas

¿Qué es?

Algo ya hemos adelantado...

Imagina que eres chef. En la cocina tienes herramientas básicas (como las funciones preconstruidas de Python), pero a veces necesitas crear tus propias recetas (funciones personalizadas) que podrás usar una y otra vez.

Una función personalizada es como crear tu propia receta: defines un conjunto de instrucciones que podrás usar cuando quieras.

Su estructura básica es:

```
def nombre_funcion():  
    # Aquí va el código que queremos que ejecute  
    print("La función está haciendo algo")
```

Funciones personalizadas

Parámetros de una función y retorno

Imagina que tienes una máquina de jugos:

- La máquina es la función.
- Las frutas que le pones son los parámetros.
- El jugo que sale es el retorno.

- Los parámetros son como los ingredientes de una receta: información que le pasamos a la función para que trabaje con ella.
- El retorno es el resultado que la función devuelve después de procesar los datos.

Funciones personalizadas

¿Qué hace la palabra clave return?

La palabra clave return se utiliza para devolver un valor desde una función.

Cuando Python encuentra un return:

- Detiene la ejecución de la función.
- Envía el valor indicado como resultado.

Ejemplo:

```
def sumar(a, b):  
    resultado = a + b  
    return resultado  
total = sumar(3, 5)  
print(total) # Imprime 8
```

Funciones personalizadas

Funciones sin parámetros

- Son como una receta que siempre se hace igual. Por ejemplo, una máquina de café que solo hace café negro:
 - No necesitan información adicional.
 - Siempre ejecutan las mismas instrucciones.
 - Útiles para tareas repetitivas que no varían.

```
def mostrar_menu():  
    """  
    Esta función muestra un menú de opciones.  
    No necesita parámetros porque siempre muestra lo mismo.  
    """  
  
    print("=== MENÚ PRINCIPAL ===")  
    print("1. Ver datos")  
    print("2. Ingresar datos")  
    print("3. Salir")  
    print("=====")
```


Funciones personalizadas

Funciones con un parámetro

- Son como una máquina que necesita una instrucción para funcionar. Por ejemplo, una tostadora que necesita saber cuánto tiempo tostar.
 - Reciben un solo dato.
 - El comportamiento varía según el parámetro.
 - Útiles para personalizar una acción simple.

```
def verificar_edad(edad):  
    """  
    Esta función verifica si una persona es mayor de edad.  
    Parámetro:  
        edad: número entero que representa la edad de la persona  
    """  
    if edad >= 18:  
        print("Puede ingresar al sitio")  
    else:  
        print("Acceso denegado - Solo mayores de edad")
```

Funciones personalizadas

Funciones con múltiples parámetros

- Son como una máquina de café moderna que puede ajustar varios aspectos. Por ejemplo, una que permite elegir tipo de café, tamaño y temperatura.
 - Reciben dos o más parámetros.
 - Permiten personalizar múltiples aspectos.
 - El orden de los parámetros es importante.

```
def calcular_imc(peso, altura, nombre):  
    """  
    Calcula el índice de masa corporal de una persona.  
    Parámetros:  
        peso: peso en kilogramos  
        altura: altura en metros  
        nombre: nombre de la persona  
    """  
  
    imc = peso / (altura ** 2)  
    imc_redondeado = round(imc, 1)  
  
    print(f"\nResultados para {nombre}:")  
    print(f"IMC: {imc_redondeado}")  
  
    if imc < 18.5:  
        print("Estado: Bajo peso")  
    elif imc < 25:  
        print("Estado: Peso normal")  
    else:  
        print("Estado: Sobrepeso")
```

Funciones personalizadas

Funciones con parámetros predeterminados

- Son como un microondas que tiene tiempos preestablecidos pero que puedes modificar. Por ejemplo, una función para calentar comida.
 - Algunos parámetros tienen valores predeterminados.
 - Los parámetros con valores por defecto van al final.
 - Se pueden omitir los parámetros opcionales.

```
def calentar_comida(alimento, tiempo=1, potencia="media"):
    """
    Simula calentar comida en microondas.
    Parámetros:
        alimento: qué vamos a calentar
        tiempo: minutos de calentado (por defecto 1)
        potencia: nivel de potencia (por defecto "media")
    """

    print("\n=== CALENTANDO COMIDA ===")
    print(f"Alimento: {alimento}")
    print(f"Tiempo: {tiempo} minutos")
    print(f"Potencia: {potencia}")
```

Funciones personalizadas

Funciones con parámetros predeterminados

```
# Recomendaciones según el tiempo
if tiempo > 3:
    print("Precaución: Tiempo prolongado")

# Recomendaciones según potencia
if potencia == "alta":
    print("¡Cuidado! Usar plato adecuado")

# Diferentes formas de uso
# 1. Solo con alimento (usa valores por defecto)
calentar_comida("Sopa")

# 2. Con alimento y tiempo
calentar_comida("Pizza", 2)

# 3. Todos los parámetros
calentar_comida("Lasaña", 3, "alta")

# 4. Especificando solo algunos parámetros
calentar_comida("Arroz", potencia="baja")
```

Funciones personalizadas

Funciones con parámetros nombrados

- Son como llenar un formulario donde cada campo tiene su nombre. Por ejemplo, un sistema de reservas.
 - Los parámetros se pueden especificar por nombre.
 - El orden no importa cuando se usan nombres.
 - Mejora la legibilidad del código
 - Especialmente útil cuando hay muchos parámetros.

Funciones personalizadas

Funciones con parámetros nombrados

```
def registrar_usuario(nombre, edad, ciudad="Santiago", ocupacion="estudiante"):
    """
    Registra un nuevo usuario en el sistema.
    Parámetros:
        nombre: nombre del usuario
        edad: edad del usuario
        ciudad: ciudad de residencia (opcional)
        ocupacion: ocupación actual (opcional)
    """
    print("\n=== DATOS DE REGISTRO ===")
    print(f"Nombre: {nombre}")
    print(f"Edad: {edad}")
    print(f"Ciudad: {ciudad}")
    print(f"Ocupación: {ocupacion}")

# Uso de la función
registrar_usuario("Juan", 25) # Usando el orden normal de parámetros
registrar_usuario(edad=30, nombre="Ana", ocupacion="ingeniera", ciudad="Valparaíso") # Parámetros nombrados
registrar_usuario("Carlos", 28, ocupacion="profesor") # Mezclando ambos métodos
```

Funciones personalizadas

Consejos importantes

Elección del tipo de función:

- Usa funciones sin parámetros para tareas que siempre son iguales
- Usa un parámetro cuando solo necesitas personalizar una cosa
- Usa múltiples parámetros cuando necesites más flexibilidad
- Usa parámetros por defecto para opciones comunes
- Usa parámetros nombrados para mejorar la claridad

Buenas prácticas:

- Los nombres de los parámetros deben ser descriptivos
- Documenta qué hace cada parámetro
- No uses demasiados parámetros (máximo 4-5)
- Agrupa parámetros relacionados

¡Manos a la obra! Creando historias personalizadas



Creando historias personalizadas

Eres un escritor de cuentos infantiles y necesitas un programa que te ayude a crear historias personalizadas según lo que los niños elijan.

1. Descarga el archivo Creador de historias mágicas.ipynb
2. Realiza cada una de las actividades que se solicitan.



/* Módulos */

Módulos

¿Qué son y para qué sirven?

Imagina que eres un chef  y tienes una cocina enorme. En ella hay:

- Un cajón solo para cuchillos 
- Otro para utensilios de repostería 
- Otro para especias 

En Python, los módulos funcionan igual:

- Son como cajones con herramientas específicas.
- Cada módulo tiene funciones para tareas específicas.
- ¡Puedes usar lo que necesites cuando lo necesites!

La librería standard de python

Imagina que te dan una tarjeta de membresía para la biblioteca más completa de tu ciudad. Esta biblioteca tiene:

- Miles de libros organizados por temas.
- Están siempre disponibles.
- ¡Y son totalmente gratis!

Así es la librería estándar de Python:

- Viene instalada automáticamente con Python.
- Tiene módulos para casi todo lo que necesites.
- ¡Y puedes usarla cuando quieras!

Módulos

Importación

Imagina que tienes una caja de herramientas. Hay diferentes formas de usarla:

- Puedes sacar toda la caja.
- Puedes sacar solo las herramientas que necesitas.
- Puedes ponerle un nombre más corto a tus herramientas favoritas.

En Python, podemos importar módulos de formas similares:

- Importación completa: Es como sacar toda la caja de herramientas.
 - Debemos usar "math." antes de cada función.
- Importación específica: Es como sacar solo las herramientas que necesitamos.
 - No necesitamos escribir "math." cada vez.
- Importación con alias: Es como ponerle un nombre más corto a nuestra caja de herramientas.

Módulos

El módulo math y stats

El módulo math tiene muchas herramientas matemáticas. Vamos a explorar las más útiles:

- pi: el número π
- sqrt(): para raíces cuadradas
- pow(): para potencias

El módulo statistics nos ayuda a analizar conjuntos de datos. Es como tener un asistente que nos ayuda a entender números.

Funciones principales:

- mean(): calcula el promedio
- median(): encuentra el valor del medio
- mode(): encuentra el valor que más se repite

¡Manos a la obra! Trabajando con módulos



Trabajando con módulos

1. Descarga el archivo módulos.ipynb
2. Realiza cada una de las actividades que se solicitan.



Resumen

Subtítulo

- Python nos ofrece dos herramientas fundamentales para organizar y reutilizar nuestro código: las funciones y los módulos.
- Las funciones actúan como recetas que podemos usar repetidamente, permitiéndonos realizar tareas específicas con diferentes datos de entrada. Estas pueden ser preconstruidas, como las que vienen incluidas en Python (`int()`, `float()`, `str()`, `type()`, `len()`, `round()`, `abs()`, `max()`, `min()`), o personalizadas, que creamos nosotros mismos según nuestras necesidades específicas.
- Las funciones personalizadas se crean usando la palabra clave `'def'` y pueden tener diferentes configuraciones: sin parámetros para tareas que siempre son iguales, con un parámetro cuando necesitamos personalizar una cosa, con múltiples parámetros para mayor flexibilidad, con parámetros predeterminados para opciones comunes, y con parámetros nombrados para mejorar la claridad del código.
- Por otro lado, los módulos son como bibliotecas especializadas que contienen conjuntos de funciones relacionadas. Python incluye una librería estándar con módulos preinstalados, como `math` para operaciones matemáticas y `statistics` para análisis

Reflexiona sobre lo aprendido hoy: Si
tuvieras que explicarle a alguien la
diferencia entre una función preconstruida
y un módulo en Python, ¿cómo lo harías?
¿Qué ejemplos de la vida cotidiana
utilizarías para que sea más fácil de
entender?"





Próxima sesión...

- *Utilizar estructuras de datos para resolver un problema según lenguaje Python.*

{desafío}
latam_

*Academia de
talentos digitales*

